IC-253 (DSA) Assignment 1 Report

Aryan Sisodiya

March 4, 2026

1 Task 1: Region Detection and Classification

1.1 Problem Description

In this task, an image is represented as a two-dimensional matrix of pixel intensities. The goal is to detect connected regions in the image and classify them as rectangular or circular.

1.2 Approach

The algorithm operates as follows:

1. Traverse the image matrix to find unvisited foreground pixels.
2. Treat such a pixel as the starting seed for a new region.
3. Expand the region along the 4 possible neighbour directions, and if any of the neighbour point satisfies all the given below conditions, push the point into the stack for possible expansion around this point.
4. Neighboring pixels are added to the region if they:
   - lie within valid matrix bounds,
   - have not been visited,
   - satisfy the pixel intensity tolerance constraint.
5. Iterate above step until the stack is empty.

During region expansion, a struct Region records the following information:

- total number of pixels in the region (area),
- bounding box limit pixels,
- cumulative coordinates of pixels.
- an array of all the pixels in that region.

1.3 Region Classification

Two classification rules are applied.

Rectangular regions: The bounding box of the region is computed. If the region nearly fills this box ($\geq 95\%$), the shape is classified as rectangular using the fill ratio:

$$\text{Fill Ratio} = \frac{\text{Region Area}}{\text{Bounding Box Area}}$$

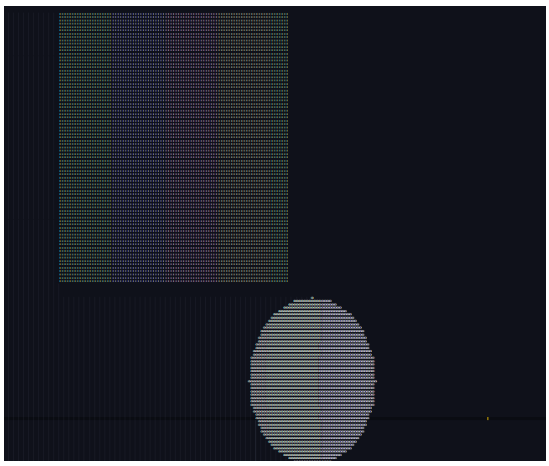
Circular regions: For circular shapes, the bounding box must be approximately square and the fill ratio should be close to the theoretical ratio of a circle inside a square (approximately 0.785), the criteria chosen is 65% < Fill Ratio < 90%

1.4 Test Case Demonstrations

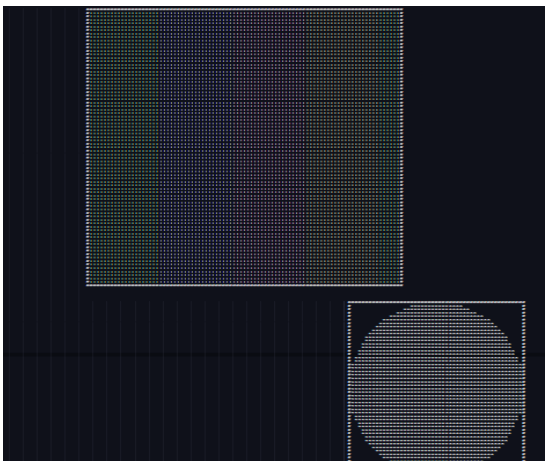
Note on Output Representation

The output generated by the Task 1 C++ program consists of three components. The textual statements indicating the classification of each detected region in the form *"Region is Circular"* or *"Region is Rectangular"*, the modified two–dimensional image matrix in which the detected region boundaries are highlighted, and an additional visualization is produced in the form of a heatmap based on pixel intensity values to provide a clearer visual interpretation of the detected structures.

For the purpose of presentation in this report, only the heatmap visualization has been included in the following test case figures due to the huge size of overall output generated. However, the complete textual output generated by the program can be seen at the GitHub repository referenced at the end of this report.

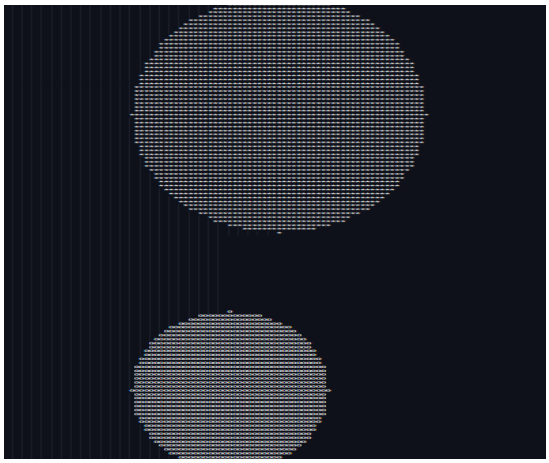


(a) Input Image

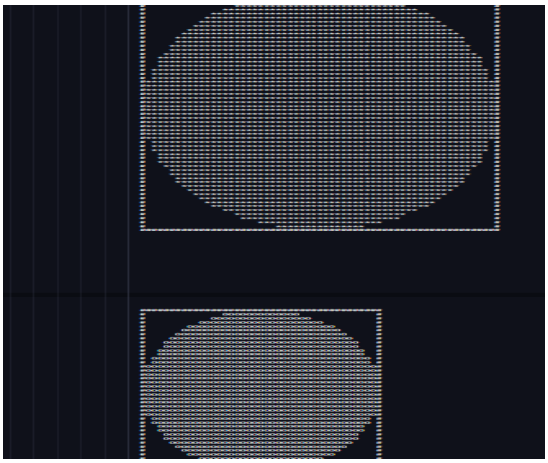


(b) Detected Regions

Figure 1: Task 1 – Test Case 1



(a) Input Image



(b) Detected Regions

Figure 2: Task 1 – Test Case 2

1.5 Time and Space Complexity Analysis

Let the image matrix have dimensions $M \times N$.

Time Complexity

The algorithm performs a traversal of the entire matrix to identify potential seed pixels for region expansion. Each pixel is visited at most once during the region growing process because a visited matrix is maintained to prevent repeated exploration.

The region expansion uses an iterative algorithm implemented with an explicit stack. Since each pixel can enter the stack at most once and each neighbor is checked a constant number of times, the total work done across all regions remains linear in the number of pixels.

Therefore, the overall time complexity is:

$$O(M \times N)$$

Space Complexity

Several auxiliary data structures are used during execution:

- The visited matrix of size $M \times N$
- A stack used for expansion (in the worst case containing $M \times N$ pixels)
- Storage for region structure definition

Thus, the total auxiliary space required is proportional to the number of pixels in the image:

$$O(M \times N)$$

2 Task 2: Sparse Matrix Representation and Object Detection

2.1 Problem Description

In this task, the image is represented as a binary matrix where:

- 1 represents foreground pixels
- 0 represents background pixels

The objective is to perform efficient image analysis using a sparse matrix representation.

The operations include:

1. constructing a sparse linked list representation,
2. identifying connected components,
3. computing the area of each object,
4. detecting boundary pixels,
5. flipping foreground and background pixels,
6. reconstructing the processed image.

2.2 Sparse Representation

Only non-zero pixel points, representing Foreground pixels are stored using a singly linked list. Each node contains:

- row index
- column index
- pixel value
- pointer to the next node

2.3 Connected Component Detection

The algorithm proceeds as follows:

1. Traverse the sparse linked list to obtain foreground pixel coordinates.
2. If a pixel has not been visited, it marks the start of a new object.
3. Perform region expansion using an explicit stack. (The same way we did in task 1).
4. Count the pixels belonging to the object.
5. Store the boundary pixel, if detected any.

2.4 Boundary Pixel Detection

A pixel is classified as a boundary pixel if any of its four neighbors:

- lies outside the matrix bounds, or
- corresponds to a background pixel.

All such pixels are stored for each detected object.

2.5 The Region Statistics:

A region Object records the following information:

1. Total pixels in that object (area).
2. Total boundary pixels detected.
3. An array of all the boundary pixels detected in the object.

2.6 Foreground–Background Flipping

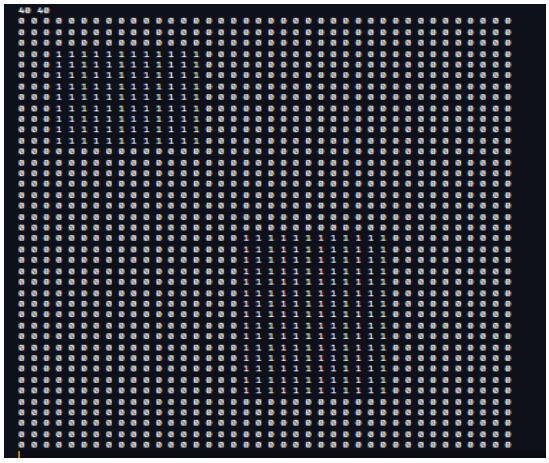
To flip the image efficiently using the sparse structure:

1. A temporary matrix marks original foreground pixels.
2. The existing sparse structure is cleared.
3. New sparse nodes are created for the complement pixels.
4. The image matrix is reconstructed from the updated sparse representation.

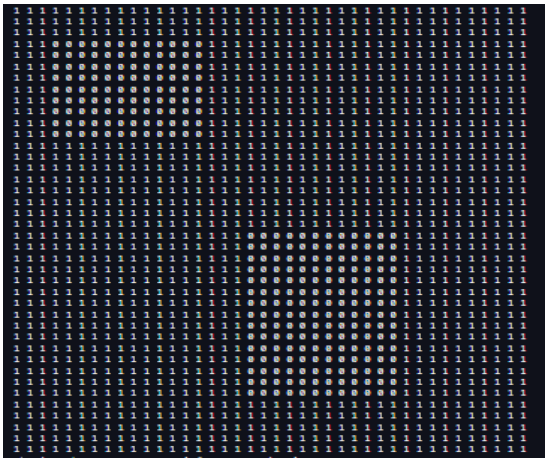
2.7 Test Case Demonstrations

Note on Output Representation

The output generated by the Task 2 C++ program contains several information printed sequentially. For the sake of clarity and brevity in this report, only the final reconstructed (flipped) image matrix for the test cases is presented in the first test case shown, as the overall output size is considerably larger. However, complete output is shown for the second test case (You might be required to zoom a little), where the matrix size is barely 10×10. Here, the brighter (white part) is 0, and the dark region is 1.

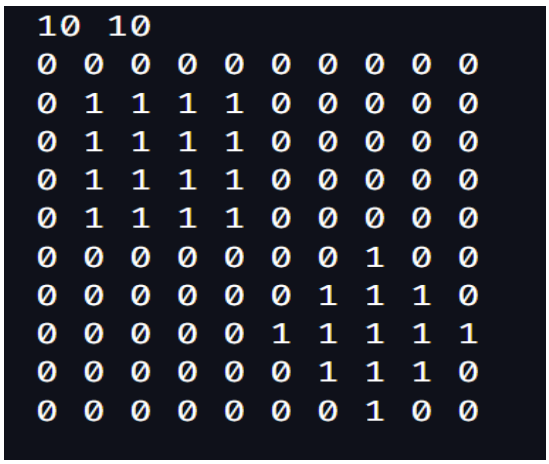


(a) Input Binary Image

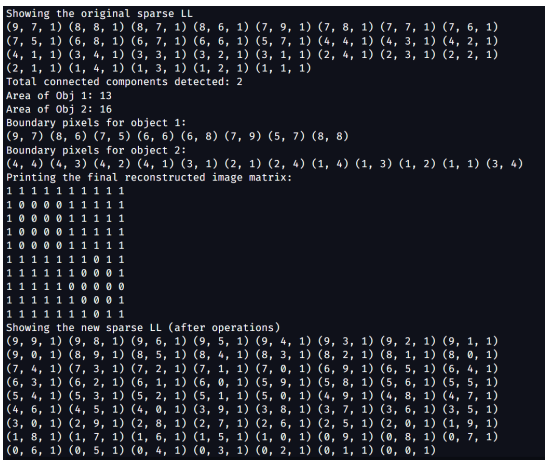


(b) Flipped Output Image

Figure 3: Task 2 – Test Case 1



(a) Input Binary Image



(b) Flipped Output Image

Figure 4: Task 2 – Test Case 2

2.8 Time and Space Complexity Analysis

Let the binary image have dimensions $M \times N$, and let K denote the number of foreground pixels stored in the sparse representation.

Time Complexity

Constructing the sparse linked list requires a single traversal of the image matrix, which takes $O(M \times N)$ time.

Connected component detection is performed using iterative algorithm. Since each foreground pixel is visited at most once and each neighbor is checked a constant number of times, the overall complexity of component detection remains linear in the number of pixels.

Flipping foreground and background pixels involves reconstructing the sparse representation by scanning the matrix and inserting the complement elements, which also requires $O(M \times N)$ time.

Reconstructing the final image matrix from the sparse representation takes $O(K)$ time.

Therefore, the overall time complexity of the complete process is dominated by matrix traversal:

$$O(M \times N)$$

Space Complexity

The algorithm uses several auxillary space:

- The visited matrix of size $M \times N$
- A sparse linked list storing K foreground pixels
- A stack used for object expansion
- Temporary storage for object and Point structure definition

In the worst case, the number of foreground pixels may approach the total number of pixels in the matrix. Therefore, the overall space complexity is:

$$O(M \times N)$$

3 Code Repository

The complete source code, along with test cases input and output files for both tasks is available in the following repository:

Task 1: Task 1 GitHub

Task 2: Task 2 GitHub